Category	Command	Description
Setup & Config	`git config --global user.name "Your Name"` `git config --global user.email "your@email.com"`	Configures author tracking credentials.
	`git init`	Initializes a brand new local repository.
	`git clone <gitlab-repo-url>`	Downloads an existing project from GitLab.
Local Changes	`git status`	Shows modified, untracked, and staged files.
	`git add <file>` or `git add .`	Stages files for the next commit snapshot.
	`git commit -m "Your message"`	Permanently saves staged changes to local history.
	`git diff`	Shows unstaged line-by-line file changes.
Branching	`git checkout -b <branch-name>`	Creates a new branch and immediately switches to it.
	`git checkout <branch-name>`	Switches over to an existing branch.
	`git branch -a`	Lists all local and remote tracking branches.
Syncing Remotes	`git remote add origin <gitlab-url>`	Links your local folder to a newly created GitLab repo.
	`git pull origin <branch-name>`	Fetches and merges the latest updates from GitLab.
	`git push -u origin <branch-name>`	Pushes your local commits up to the GitLab repository.

---

2. GitLab CLI Commands (`glab`)

The official GitLab CLI (`glab`) brings your GitLab web interface directly into your shell environment. [1]

- **`glab auth login`** – Authenticates your terminal session with your GitLab instance account.
- **`glab mr create`** – Creates a Merge Request (MR) straight from your current development branch.
- **`glab mr list`** – Lists active merge requests tracking across the project repository.
- **`glab issue create`** – Opens a new project tracking bug or feature issue ticket.
- **`glab issue list`** – Displays active filtered issues assigned to your team.
- **`glab ci view`** – Monitors your active GitLab CI/CD pipelines and pipeline logs directly.

- **`glab repo clone <path>`** – A shortcut command syntax to clone individual projects natively.